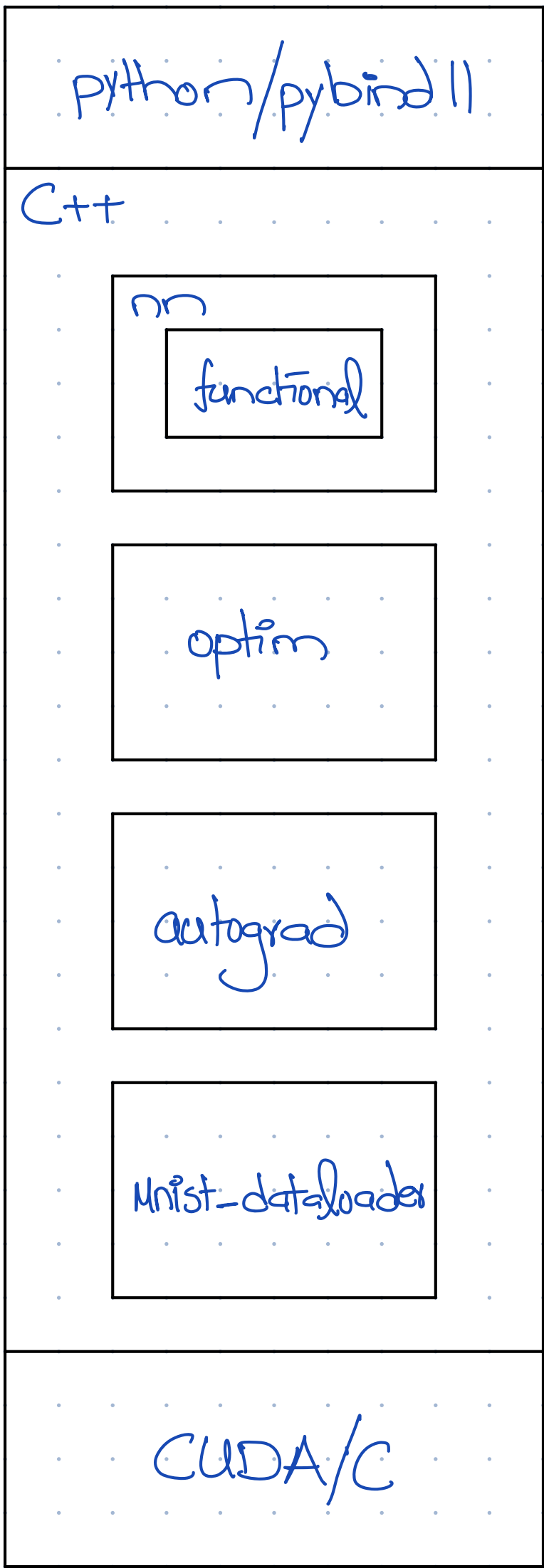


Architecture



Tensor

```
typedef struct Tensor {  
    float *data; // pointer to data  
    int ndim;    // number of dimensions of the data  
    int *shape;  // shape of data  
    int size;    // total size of data  
    Node *prev;  // pointer to a node  
  
    int on_gpu;  // flag that represents whether Tensor is on gpu or cpu  
    int requires_grad; // flag that tells whether this Tensor gradient  
    struct Tensor *grad; // A gradient Tensor, created only if requires_grad is true  
} Tensor;
```

- We keep the meta data (ndim, shape, size) always on cpu
- When utility function `tensor-to-gpu(...)` or `tensor-to-cpu(...)` is called only data & grad → data is moved.

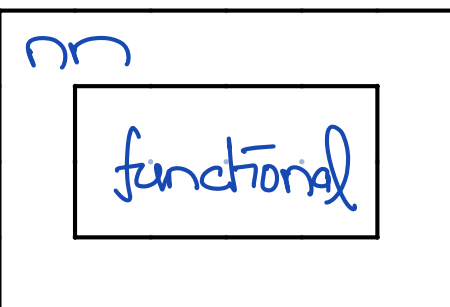
This is done as kernels are passed necessary metadata by value, so the cuda cores don't interact with CPU RAM at all.

Node

```
typedef void (*backward_fn)(Node *node, Tensor *dout);  
  
typedef struct Node {  
    Tensor **inputs; // array of tensors that were used to create this node  
    void **ctx;      // extra context needed needed  
  
    int num_inputs;  
    int num_ctx;  
  
    // Generic func pointer  
    backward_fn backward;  
} Node;
```

→ we don't need to pass node as dout has access to it, we'll leave to future refactor

- A generic backward function pointer is created because each backward kernel needs different parameters & cpp doesn't allow this directly.



→ stateful

class Conv2D:

[Weights]

input passed (eg: image)

forward(input: Tensor)

show src/nn/conv.cpp

conv2d-functional-forward(...)

→ Actually does the computational

weights

output

input

- Logical-Or: if either inputs requires grad, the output also requires grad

Show src/functional/conv.cpp

```
(input->requires_grad || this->requires_grad)
```

weights

- A generic function pointer selecting computes grad for inputs & weights according

- The remaining layers follow this same logic
(show all remaining files)

optim

class SGD:

[list of all layer weights]

[lr]

zero_grad()

↳ zeroes out all grads

step()

weight.data $\leftarrow$ weight.data - lr * weight.grad

↳ loops over each weight in the list & calls a cuda kernel that executes this

show cuda/sgd.cu

→ Tensors

show src/optim/sgd.cpp

autograd

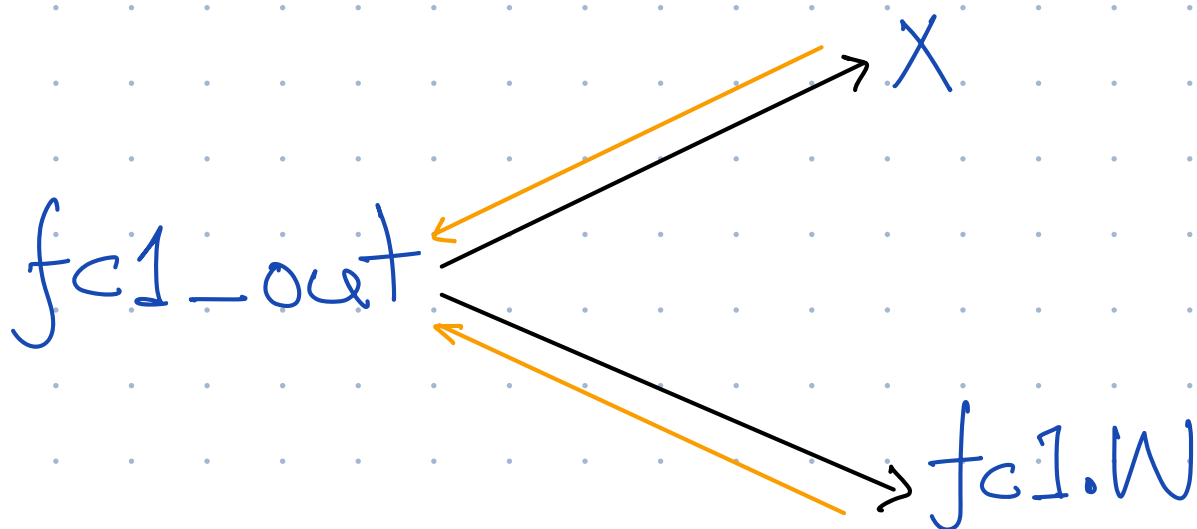
Let's define a small network:

```
# Layer Definitions (Allocates GPU State)
fc1 = nn.Linear(in_features=in_features, out_features=hidden_features, has_bias=0, requires_grad=1)
relu = nn.ReLU(requires_grad=1)
fc2 = nn.Linear(in_features=hidden_features, out_features=10, has_bias=0, requires_grad=1)
ce = nn.CrossEntropy(requires_grad=1)
```

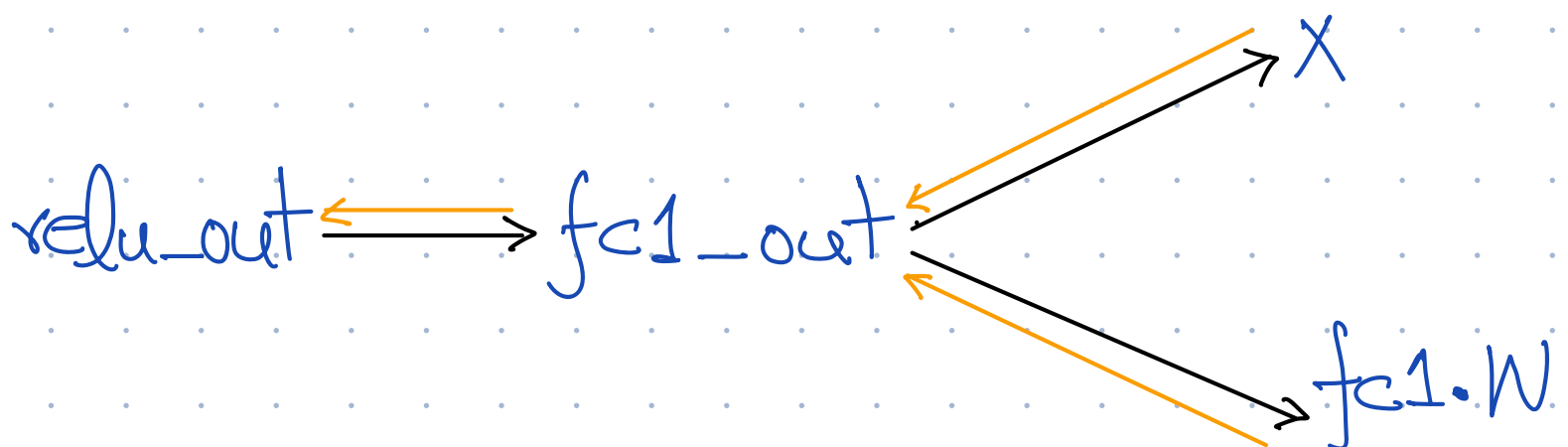
Building the graph layer by layer

```
fc1_out = fc1.forward(x)
```

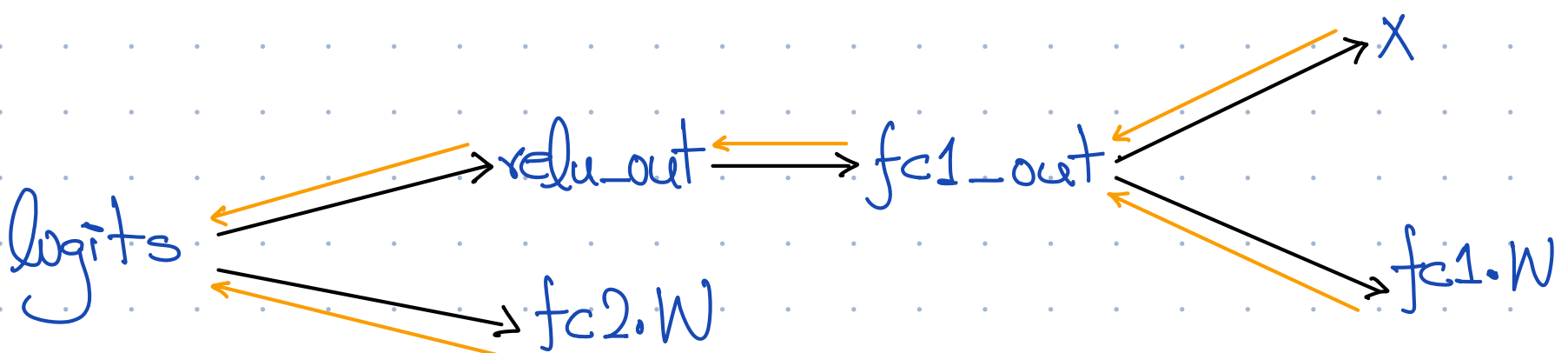
→ forward pass
→ backward pass



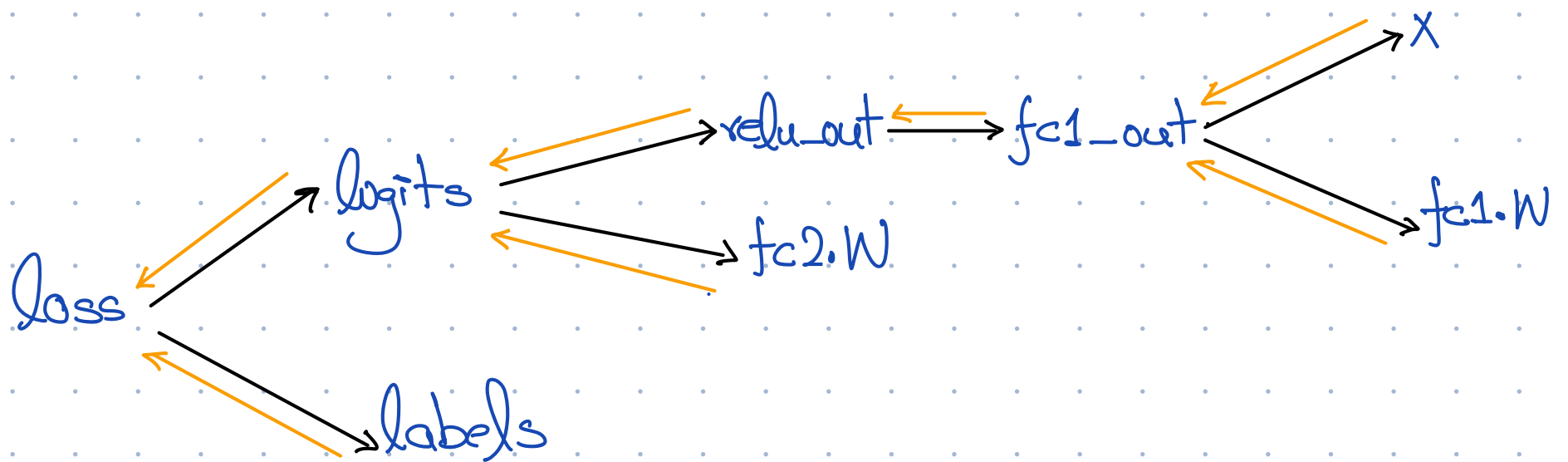
```
relu_out = relu.forward(fc1_out)
```



```
logits = fc2.forward(relu_out)
```



```
loss = ce.forward(logits, labels)
```



```
optim.backward(loss)
```

↳ Toposort (computation graph) → compute grad

- Let vertices represent tasks. So, a directed edge $\text{relu_out} \rightarrow \text{fc1_out}$ represents that gradient of relu_out must be computed before gradient of fc1_out
- We can achieve this reversing the result of Dfs post-order

↳ explore all children of a node Δ then add them to the stack

↳ backward

- Calls the generic backward on each tensor

$t \rightarrow \text{prev} \rightarrow \text{backward}(t \rightarrow \text{prev}, t \rightarrow \text{grad})$ ↑ incoming grad

show src/autograd/backward.cpp

visited = [loss, logits, relu-out, fc1-out, x, weights]
topo = [fc1-out, relu-out, logits, loss]

reverse topo:

loss, logits, relu-out, fc1-out

Mnist-dataloader

- Loads the entire dataset alongside labels into RAM as it is pretty small.

$60000 \times (28 \times 28) \times 1$ bytes

47,040,000 bytes

45MB

show mnist-dataloader/mnist.cpp

python/pybind11

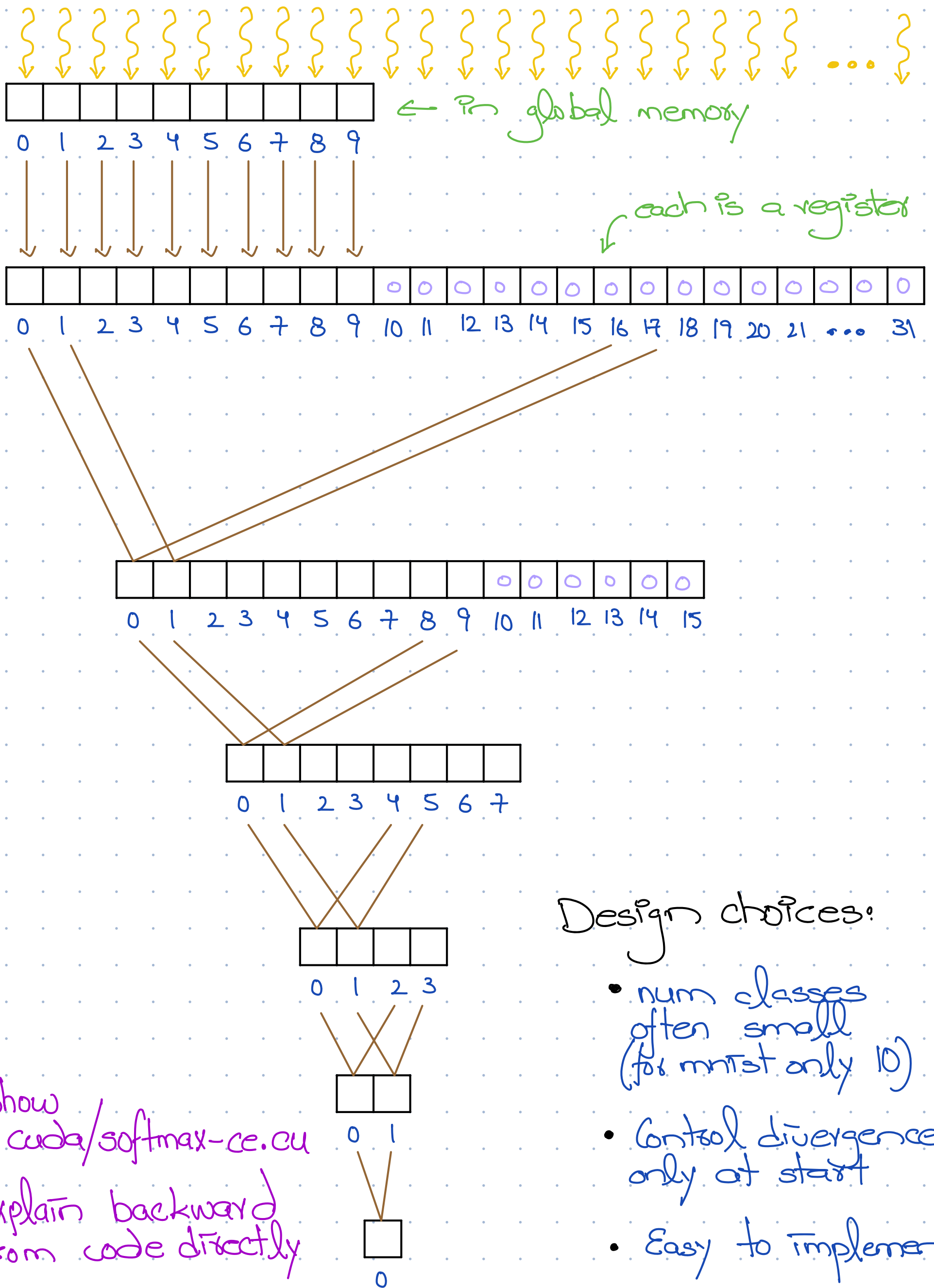
show bind/

Limitations

We have an "adoption" policy because we do not leave any orphaned tensors behind. When a layer returns a newly allocated tensor, you effectively "adopt" it and assume full responsibility for its life cycle. Concretely, you must manually free all tensors using the utility function `base.tensor_free(Tensor *)`. A simple destructor is insufficient here because it cannot account for the complex dependencies within the computational graph. A proper automated solution would require a dedicated garbage collector, which is out of scope for this project. Now you have seen manual memory management in Python (go ahead, laugh).

CUDA/C

softmax_ce:



Design choices:

- num classes often small (for mnist only 10)
- Control divergence only at start
- Easy to implement

show
cuda/softmax-ce.cu
explain backward
from code directly

Fused softmax kernel

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_i e^{z_i}}$$

$$\mathcal{L} = - \sum_{i=1}^C y_i \ln(\sigma(z_i))$$

$$= - \sum_{i=1}^C y_i \ln \left[\frac{e^{z_i}}{\sum_i e^{z_i}} \right]$$

$$= - \sum_{i=1}^C y_i \left[\ln(e^{z_i}) - \ln \left(\sum_i e^{z_i} \right) \right]$$

$$= - \sum_{i=1}^C y_i \left[z_i - \ln \left(\sum_i e^{z_i} \right) \right]$$

As y_i is label index $[1, 2, 0 \dots]$ we can simplify

$$= -z_k + \ln \left(\sum_i e^{z_i} \right)$$

For numerical stability, we do:

$$= -(z_k - M) + \ln \left(\sum_i e^{z_i - M} \right)$$

where M is $\max_C z$

For backward pass,

$$\mathcal{L} = -z_k + \ln \left(\sum_i e^{z_i} \right)$$

Case 01: target logit

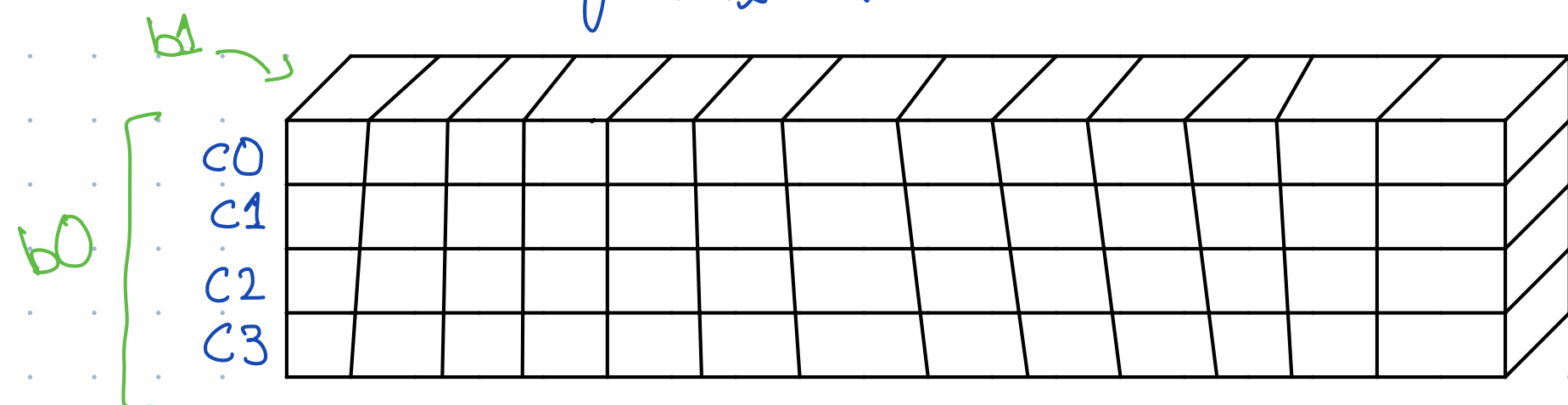
$$\frac{dL}{dz_k} = -1 + \frac{e^{z_i}}{\sum_i e^{z_i}}$$

Case 01: not target logit

$$\frac{dL}{z_{j \neq k}} = 0 + \frac{e^{z_i}}{\sum_i e^{z_i}}$$

global-pooling:

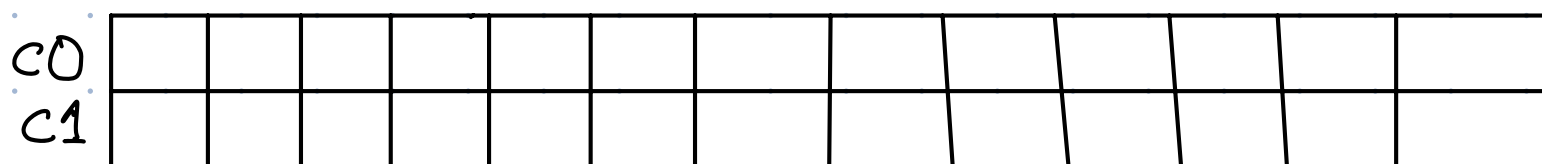
- As we perform reduction over HxW dim, we consider the data as 1D instead of 2D as the result is equivalent



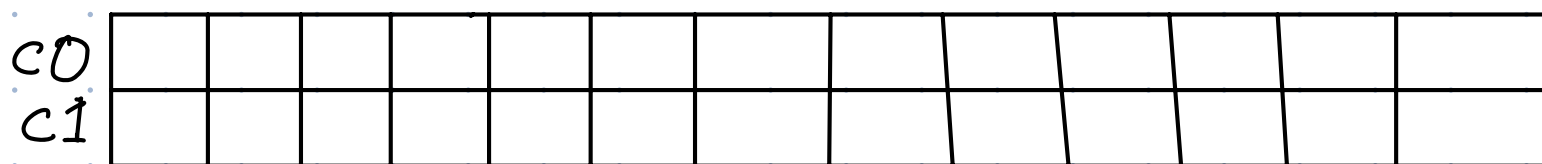
Assuming for HxW, we have only block, CF 2
blocks launched = $\frac{2}{2} * \frac{4}{2} = 2$

```
int channel_groups = cdiv(num_channels, COARSE_FACTOR_FORWARD);  
int start_channel_idx = (blockIdx.x % channel_groups) * COARSE_FACTOR_FORWARD;  
int start_batch_idx = (blockIdx.x / channel_groups) * COARSE_FACTOR_FORWARD;
```

b0:



b1:



block 0
works
on this

$$\text{channel_grps} = 4/2 = 2$$

$$\text{start_batch_idx} = 0/2 * 2 \quad [0]$$

$$\text{start_channel_idx} = 0 \% 2 * 2 \quad [0]$$

batch loops from 0-1

channel loops from 0-1

perform reduction over flattened data at
batch b & channel c

b0:

c2												
c3												

b1:

c2												
c3												

block 1
works
on this

$$\text{channel_grps} = 4/2 = 2$$

$$\text{start_batch_idx} = 1/2 * 2 \quad [0]$$

$$\text{start_channel_idx} = 1 \% 2 * 2 \quad [2]$$

batch loops from 0-1 → batch loop same

channel loops from 2-3 → channels shifted

perform reduction over flattened data at
batch b & channel c

Another Example, with 3 batches & 4 channels

$$\text{blocks launched} = \lceil 3/2 \rceil * \lceil 4/2 \rceil = 4$$

blockIdx : batch : channel

0 0-1 0-1

1 0-1 2-3

2 2-3 0-1

3 2-3 2-3

$$(0/2)*2, (0\%2)*2$$

$$(1/2)*2, (1\%2)*2$$

$$(2/2)*2, (2\%2)*2$$

$$(3/2)*2, (3\%2)*2$$

↳ skipped as
out of bounds
(line 22)

show cuda/global-pooling.cu

explain backward from code directly

Convolution 2D:

Forward Pass

- Same Coarsing over batch & in-channels is done here.

For $H, W: (16, 16)$ kernel-size = 3, blockDim (8, 8)

- Threads are launched according to output data

$$\text{pad-h} = 3 - 1/2 = 1$$

$$\text{pad-w} = 3 - 1/2 = 1$$

$$\text{tile_dim} = 8 + 3 - 1 = 10$$

```
// co-operative load of input tile
for (int t = (threadIdx.y * blockDim.x + threadIdx.x); t < (tile_dim * tile_dim); t += (blockDim.x * blockDim.y))
{
    int load_y = t / tile_dim;
    int load_x = t % tile_dim;

    int in_y = (blockIdx.y * blockDim.y) + load_y - pad_h;
    int in_x = (blockIdx.x * blockDim.x) + load_x - pad_w;

    if (in_y >= 0 && in_y < input_height && in_x >= 0 && in_x < input_width)
        s_input_tile[load_y * tile_dim + load_x] = in[bs * (in_channels * input_height * input_width) +
                                                    c * (input_height * input_width) + in_y * input_width + in_x];
    else
        s_input_tile[load_y * tile_dim + load_x] = 0.0f;
}
```

flatten 2D threadIdx to one
convert back to 2D
increment by all threads in the block

blockIdx.x = 0, blockIdx.y = 0

tid	load-y	load-x	in-y	in-x
0	0/10	0%10	$(0*8)+0-1=-1$	$(0*8)+0-1=-1$
1	1/10	1%10	$(0*8)+0-1=-1$	$(0*8)+1-1=0$
⋮	⋮	⋮	⋮	⋮
10	10/10	10%10	$(0*8)+0-1=-1$	$(0*8)+0-1=-1$
11	11/10	11%10	$(0*8)+1-1=0$	$(0*8)+1-1=0$
12	12/10	12%10	$(0*8)+1-1=0$	$(0*8)+2-1=1$
⋮	⋮	⋮	⋮	⋮
0+64	64/10	64%10	$(0*8)+6-1=5$	$(0*8)+4-1=3$
1+64	65/10	65%10	$(0*8)+6-1=5$	$(0*8)+5-1=4$

```
for (int j = 0; j < COARSE_FACTOR; j++)
```

```
{
```

```
    int filter_k = start_filter_idx + j;
```

```
    if (filter_k < out_channels)
```

```
    {
```

```
        for (int f_i = 0; f_i < kernel_size; f_i++)
```

```
        {
```

```
            #pragma unroll
```

```
            for (int f_j = 0; f_j < kernel_size; f_j++)
```

```
            {
```

```
                accumulators[j] += s_input_tile[(threadIdx.y + f_i) * tile_dim + (threadIdx.x + f_j)] *
```

```
                filters[filter_k * (in_channels * kernel_size * kernel_size) +
```

```
                c * (kernel_size * kernel_size) + (f_i * kernel_size) +
```

```
                f_j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

→ pick a specific out-channel value

→ multiply it KxK for each in-channel & sum them

Backward Pass for weights

- Launch threads according to `output.grad`, which has same shape as `output`.

```
__shared__ float acc_reduction[K][K];
if (threadIdx.y < K && threadIdx.x < K)
    acc_reduction[threadIdx.y][threadIdx.x] = 0.0f;

float acc[K][K];
#pragma unroll
for (int ky = 0; ky < K; ky++) {
    #pragma unroll
    for (int kx = 0; kx < K; kx++) {
        acc[ky][kx] = 0.0f;
    }
}
```

- Similarly, load input tile into shared memory

get grad value

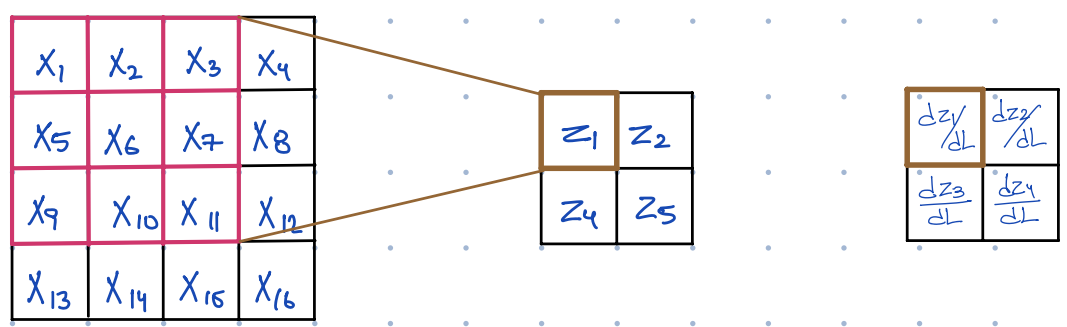
```
float dout_i = dout[bs * (out_channels * output_height * output_width) +
                    out_channel_idx * (output_height * output_width) +
                    out_y * (output_width) +
                    out_x];

#pragma unroll
for (int f_i = 0; f_i < K; f_i++)
{
    #pragma unroll
    for (int f_j = 0; f_j < K; f_j++)
    {
        acc[f_i][f_j] += s_input_tile[(threadIdx.y + f_i) * tile_dim + (threadIdx.x + f_j)]
                        * dout_i;
    }
}
```

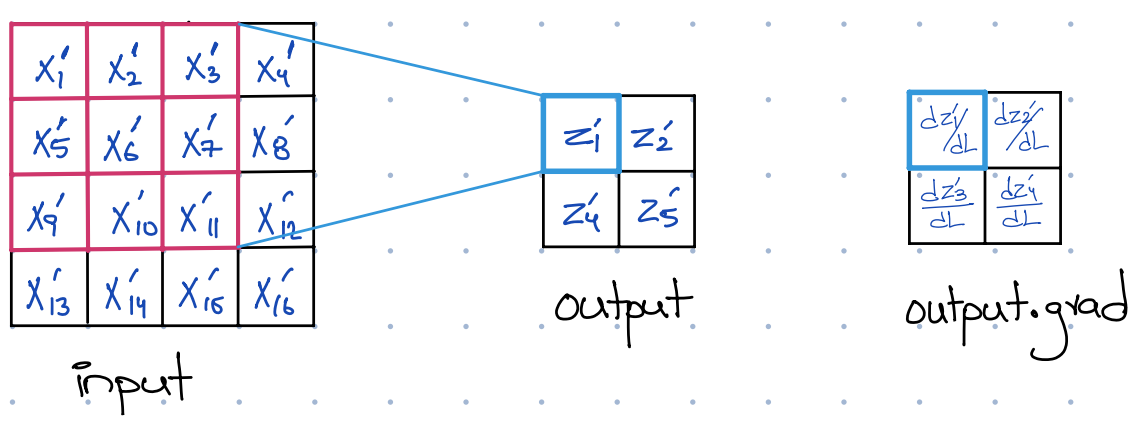
} multiply with the input $K \times K$ it was computed for

Toy Example

batch: 0



batch: 1



$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_5 & x_6 & x_7 \\ x_9 & x_{10} & x_{11} \end{bmatrix} * \frac{dz_1}{dL} + \begin{bmatrix} x_1 & x_2 & x_3 \\ x_5 & x_6 & x_7 \\ x_9 & x_{10} & x_{11} \end{bmatrix} * \frac{dz'_1}{dL}$$

Repeat this every tile & sum them all

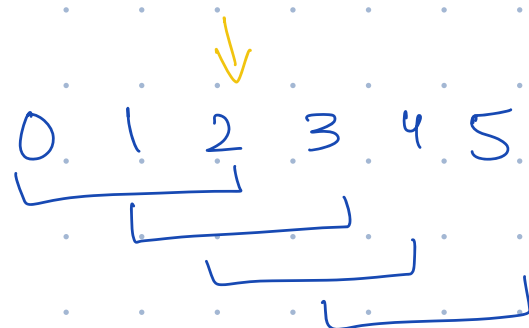
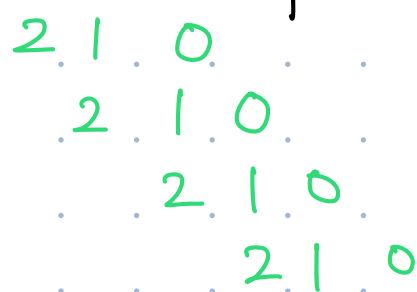
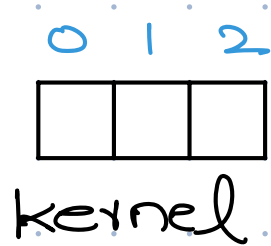
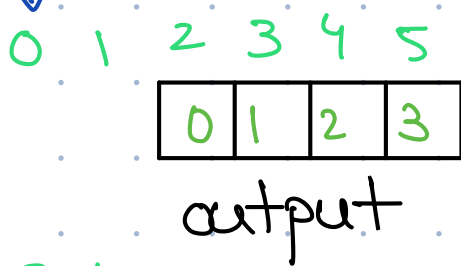
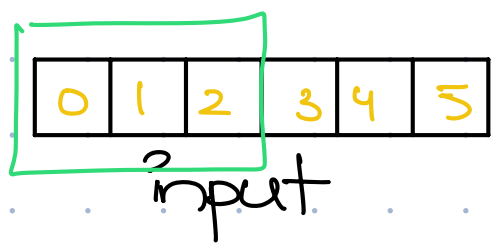
```
// Perform reduction in shared mem (block-level)
for (int ky = 0; ky < kernel_size; ky++) {
    for (int kx = 0; kx < kernel_size; kx++) {
        atomicAdd(&acc_reduction[ky][kx], acc[ky][kx]);
    }
}

__syncthreads();

// Perform reduction in global memory
if (threadIdx.y == 0 && threadIdx.x == 0) {
    for (int ky = 0; ky < kernel_size; ky++) {
        for (int kx = 0; kx < kernel_size; kx++) {
            atomicAdd(&grad_w[out_channel_idx * (in_channels * kernel_size * kernel_size) +
                in_channel_idx * (kernel_size * kernel_size) +
                ky * (kernel_size) +
                kx], acc_reduction[ky][kx]);
        }
    }
}
```

Backward pass for input

Let's look at a 1D example:



output 0: [0 1 2]

output 1: [1 2 3]

output 2: [2 3 4]

output 3: [3 4 5]

input	output
0	0
1	0, 1
2	0, 1, 2
3	1, 2, 3

So,
for input $\rightarrow$ grad[2]
I need output $\rightarrow$ grad[0, 1, 2]

So, we need to start
from 0 & load till

$\rightarrow$ start position of input tile
that kernel needs to slide over
 $\rightarrow$ no pad used in
loop

we reach 2.

Mathematically,

$$1 - (3 - 1)$$

$$\text{input_idx} - (k - 1)$$

$$\text{in_idx} = \text{out_idx} + k$$
$$(d) \text{out_idx} = \underbrace{\text{in_idx} - (k - 1)}_{\text{start pos}} - k$$

move back kernel's maximum reach

with padding

$$\text{in_idx} = \text{out_idx} - \text{pad}$$

$$(d) \text{out_idx} = \text{in_idx} + \text{pad} - (k - 1)$$

max reach

```
for (int t = (threadIdx.y * blockDim.x + threadIdx.x); t < (tile_dim * tile_dim); t += (blockDim.x * blockDim.y)) {
    int load_y = t / tile_dim;
    int load_x = t % tile_dim;

    int dout_y = ((blockIdx.y * blockDim.y) + pad_h - kernel_size + 1) + load_y; // load the lowest yth it needs
    int dout_x = ((blockIdx.x * blockDim.x) + pad_w - kernel_size + 1) + load_x; // load the lowest yth it needs

    if (dout_y >= 0 && dout_y < output_height && dout_x >= 0 && dout_x < output_width)
        s_dout_tile[load_y * tile_dim + load_x] = dout[batch_idx * (out_channels * output_height * output_width) +
            out_c * (output_height * output_width) +
            dout_y * output_width +
            dout_x];
    else
        s_dout_tile[load_y * tile_dim + load_x] = 0.0f;
}
```

start-pos for the block

- for computation, we do the same thing but drop pad_w & pad_h as it is already done while loading

$$(\text{threadIdx.x} - k + 1) - kx$$

max-reach of kernel

$$(\text{threadIdx.y} - k + 1) - ky$$

We can simplify this by reversing loop from $k-1 \rightarrow 0$

$$\text{threadIdx.x} + kx \rightarrow (k-1) - kx$$

$$\text{threadIdx.y} + ky \rightarrow (k-1) - ky$$

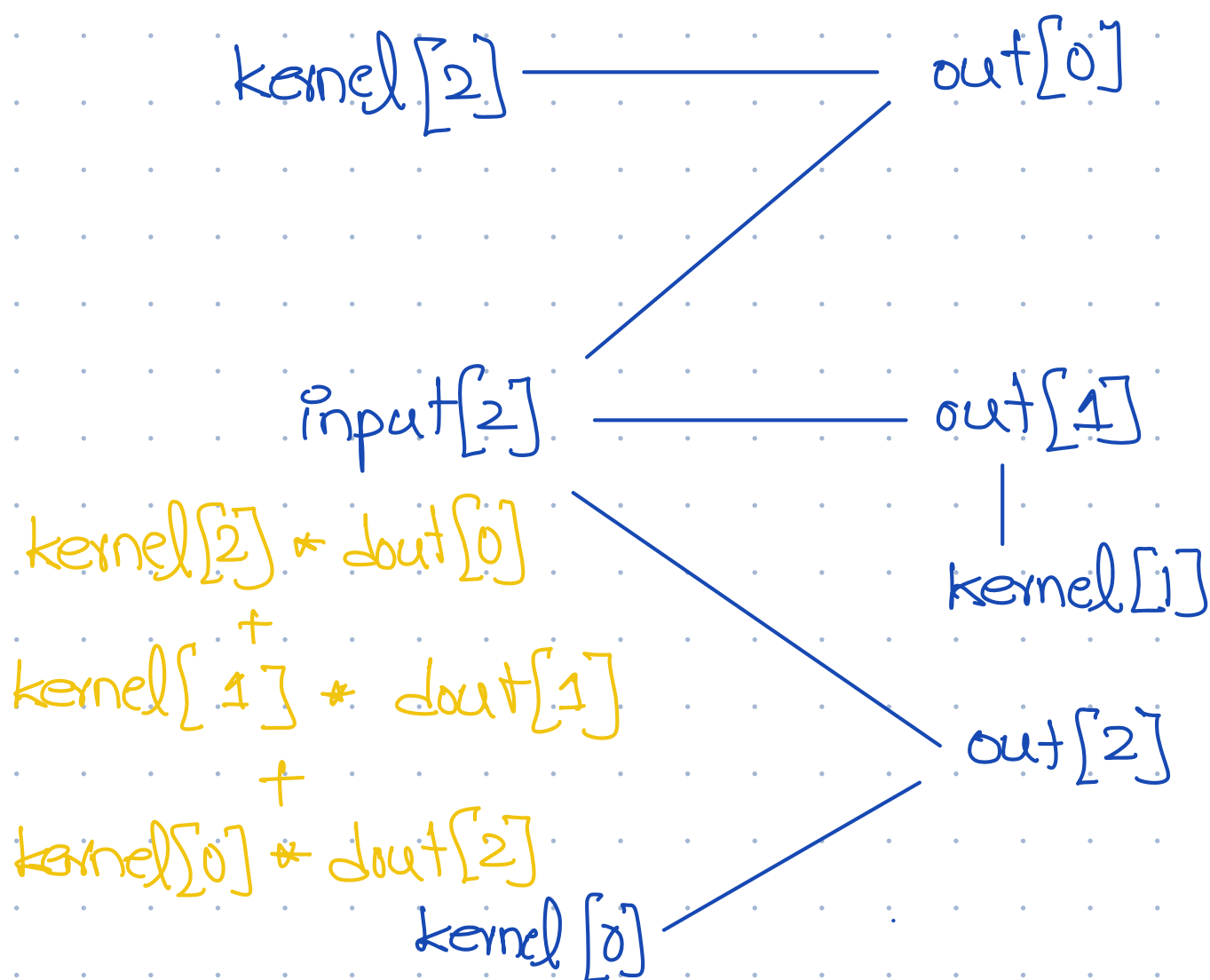
because

$$\text{out}[0] = \text{input}[1] * \text{kernel}[2]$$

$$\text{out}[1] = \text{input}[1] * \text{kernel}[1]$$

$$\text{out}[2] = \text{input}[1] * \text{kernel}[0]$$

So, $\text{input.grad}[1]$



So, following rules of backprop, gives the same thing!!

This was

Tiny torch